

Exercícios — Módulo 5: Repetição (Loops)

Aula 5 · 17/07 · Jornada DEV START — Programa START (TOTVS Paulista) Prazo sugerido: até a **Aula 6 (20/07)** — há um **fim de semana pela frente**, ótima janela para praticar com calma. 💪 **Onde entregar:** na **atividade do Módulo 5 no Google Classroom**. Suba o código no seu **GitHub** e cole o link na atividade. Depois, comemore postando o print no Discord **#conquistas**. 🎉

Antes de começar

- Todos os exercícios são em **Harbour puro**, no VSCode.
- Fluxo de sempre: escrever → `hbm2 arquivo.prg` → `arquivo.exe`.
- Antes de rodar, revise os **3 ingredientes** do laço: inicialização, condição e atualização — é o que evita o **loop infinito**. Travou? `Ctrl + C` no terminal.
- Regra de bolso: **sabe quantas vezes?** `FOR`. **Não sabe?** `WHILE`.
- Não trave sozinho(a): dúvida é no `#duvidas-parte1-harbour`. 🤝

Exercício 1 — Sequências com FOR/NEXT

Usando `FOR/NEXT`, escreva um programa que: - a) Imprima os números de **1 a 100** - b) Imprima os números de **-50 a 50** - c) Imprima os números de **80 a 5** em ordem **decrescente** (use `STEP -1`)

Exercício 2 — Loop indefinido: o dobro

Leia valores inteiros **indefinidamente** e, para cada um, exiba o **dobro** do valor lido. Pare quando o usuário informar um valor **zero ou negativo**.

Dica: não dá para saber quantos serão → use `WHILE`. Leia um valor antes do teste ou use `WHILE .T.` + `EXIT`.

Exercício 3 — Máquina de soma

Leia valores do usuário **até que o valor 0 seja digitado**. Ao final, mostre: - A **soma total** dos valores - A **quantidade** de valores somados (não conte o 0 final)

Dica: você precisa de um **acumulador** (`nTotal`) e um **contador** (`nQtd`), ambos inicializados **antes** do loop.

Exercício 4 — Validação completa (loops de consistência)

Leia **nome**, **disciplina** e **2 notas** de um aluno, validando cada campo com um loop de consistência (só avança quando o valor for válido): - **Nome:** não pode ser vazio - **Disciplina:** exatamente **3 letras maiúsculas** (ex.: `MAT` , `POR`) - **Notas:** cada uma entre **0 e 10**

Ao final, exiba os dados e a **média** das duas notas.

Dica: `Len(Trim(cNome)) > 0` verifica não-vazio; `Len(cDisc) == 3 .AND. cDisc == Upper(cDisc)` ajuda na disciplina.

Exercício 5 — Loop de programa

Faça uma versão do Exercício 4 que fique **rodando**: ao final de cada aluno, pergunte “**Deseja calcular outro? (S/N)**”. Enquanto a resposta for `S` , processe um novo aluno; caso contrário, encerre.

Dica: envolva tudo em um `WHILE .T.` e use `EXIT` quando a resposta não for `S` .

Exercício 6 — Desafio: jogo “Adivinhe o número”

Crie um jogo em que: - O programa **sorteia** um número entre 1 e 100 com `HB_RandomInt(1, 100)` - O usuário tem **7 tentativas** - A cada palpite errado, informe se o número secreto é **maior** ou **menor** - Ao **acertar** ou **esgotar** as tentativas, mostre o resultado (e revele o número)

Dica: um `FOR nI := 1 TO 7` conta as tentativas; use `EXIT` ao acertar.

Como sei que acertei?

Exercício	Está pronto quando...
1	As três sequências saem corretas, inclusive a decrescente (80...5)
2	Mostra o dobro de cada valor e para no primeiro 0 ou negativo
3	Exibe a soma total e a quantidade certa de valores (sem contar o 0)
4	Recusa nome vazio, disciplina fora do padrão e nota fora de 0–10
5	Processa vários alunos em sequência e só para quando você pede
6	Dá as dicas maior/menor e encerra ao acertar ou nas 7 tentativas

✅ Fez do 1 ao 5? Você cumpriu o essencial do módulo. O 6 é o “algo a mais” — perfeito para o fim de semana.

Como organizar no seu repositório

Salve os exercícios desta aula na pasta `modulo-05/`, um arquivo por exercício (guia completo em `pre-curso/setup-github.md`):

```
modulo-05/  
├─ ex01-sequencias-for.prg  
├─ ex02-dobro-while.prg  
├─ ex03-maquina-soma.prg  
├─ ex04-validacao-aluno.prg  
├─ ex05-loop-programa.prg  
└─ ex06-advinha-numero.prg  (desafio – opcional)
```

Depois: `git add .` → `git commit -m "modulo 05 - exercicios"` → `git push`, e cole o link da pasta na atividade do Módulo 5 no Classroom.

Guia de referência rápida

```
// Laço com pré-teste (não sabe quantas vezes)
WHILE <condição>
    // ...          ← ATUALIZE a condição (senão: loop infinito!)
ENDDO

// Laço contado (sabe quantas vezes)
FOR nI := <início> TO <fim> STEP <passo>
    // ...
NEXT

// Loop geral / validação de entrada
WHILE .T.
    // ...
    IF <condição de saída>
        EXIT
    ENDIF
ENDDO

// Controle de fluxo
EXIT          // sai do laço imediatamente
LOOP         // pula para a próxima iteração

// Ferramentas úteis
nCont++      // contador (+1)
nTotal += nValor // acumulador
Mod(nI, 2) == 0 // nI é par?
Val(cTexto)   // texto → número
Str(nNumero, 8, 2) // número → texto
Len(Trim(cTexto)) // tamanho do texto sem espaços nas pontas
Upper(cTexto) // MAIÚSCULAS
HB_RandomInt(1,100) // sorteia um inteiro de 1 a 100
```